

DPrivStats

Reference Manual — Differentially Private Classical Statistical Inference in R
Version 0.1.0 · Author: Mukul Bijalwan · License: MIT · github.com/MukulBijalwan/DPrivStats

Contents

Package Overview	DPrivStats-package, example_microdata, simulate_data
Privacy Mechanisms	rlaplace, cpp_rlaplace, laplace_mechanism, gaussian_sigma, gaussian_mechanism, analytic_gaussian_sigma, analytic_gaussian_mechanism, exponential_mechanism
DP Descriptive Statistics	stat_sensitivities, dp_mean, dp_variance, dp_quantile, dp_median, dp_histogram
DP Hypothesis Tests	dp_t_test, dp_chisq_test, dp_ks_test, asymptotic_ks_pvalue, dp_anova
DP Regression & Inference	dp_lm, dp_lm_sensitivity, summary.dp_lm, dp_glm, clip_gradients, cpp_clip_gradients, dp_confint, confint.dp_lm
Privacy Budget & Composition	new_privacy_budget, spend, can_spend, basic_composition, advanced_composition_epsilon, epsilon_to_rdp, rdp_to_epsilon, rdp_composition, laplace_plr_tail, empirical_delta_from_losses, simulate_gaussian_losses
Diagnostics	validate_coverage, compare_utility, compare_composition, dp_utility_diagnostics
Print Methods	print.dp_estimate, print.dp_htest, print.dp_lm, print.summary.dp_lm, print.dp_glm, print.dp_confint, print.privacy_budget

Package Overview

DPrivStats-package
DPrivStats: Differentially Private Classical Inference

example_microdata
Census-like example microdata

example_microdata

simulate_data

Simulate synthetic microdata

```
simulate_data(n = 1000)
```

Privacy Mechanisms

rlaplace

Sample Laplace noise

```
rlaplace(n, location = 0, scale = 1)
```

cpp_rlaplace

Fast Laplace sampling (C++)

```
cpp_rlaplace(n, scale = 1)
```

laplace_mechanism

Laplace mechanism

```
laplace_mechanism(f, x, epsilon, l1_sensitivity)
```

gaussian_sigma

Classic Gaussian noise scale

```
gaussian_sigma(epsilon, delta, l2_sensitivity)
```

gaussian_mechanism

Gaussian mechanism

```
gaussian_mechanism(f, x, epsilon, delta, l2_sensitivity)
```

analytic_gaussian_sigma

Analytic Gaussian calibration (Balle & Wang, 2018)

```
analytic_gaussian_sigma(epsilon, delta, l2_sensitivity)
```

analytic_gaussian_mechanism

Analytic Gaussian mechanism

```
analytic_gaussian_mechanism(f, x, epsilon, delta, l2_sensitivity)
```

exponential_mechanism

Exponential mechanism sampler

```
exponential_mechanism(candidates, scores, epsilon, score_sensitivity = 1)
```

DP Descriptive Statistics

stat_sensitivities

Sensitivity of common bounded-data statistics

```
stat_sensitivities(bounds, n)
```

dp_mean

DP mean

```
dp_mean(  
  x,  
  epsilon,  
  bounds = range(x, na.rm = TRUE),  
  mechanism = c("laplace", "gaussian", "analytic_gaussian"),  
  delta = NULL  
)
```

dp_variance

DP variance

```
dp_variance(  
  x,  
  epsilon,  
  bounds = range(x, na.rm = TRUE),  
  mechanism = c("laplace", "gaussian"),  
  delta = NULL  
)
```

dp_quantile

DP quantile via the exponential mechanism

```
dp_quantile(x, epsilon, bounds, probs = 0.5, n_bins = 1000)
```

dp_median

DP median via the exponential mechanism

```
dp_median(x, epsilon, bounds, n_bins = 1000)
```

dp_histogram

DP histogram

```
dp_histogram(x, epsilon, breaks = NULL, normalize = FALSE)
```

DP Hypothesis Tests

dp_t_test

DP two-sample t-test

```
dp_t_test(  
  x,  
  y,  
  epsilon,  
  bounds = c(min(c(x, y), na.rm = TRUE), max(c(x, y), na.rm = TRUE)),  
  alternative = c("two.sided", "less", "greater"),  
  delta = NULL  
)
```

dp_chisq_test

DP chi-square test of independence

```
dp_chisq_test(table, epsilon)
```

dp_ks_test

DP Kolmogorov-Smirnov test

```
dp_ks_test(  
  x,  
  y,  
  epsilon,  
  bounds = c(min(c(x, y), na.rm = TRUE), max(c(x, y), na.rm = TRUE)),  
  n_grid = 100  
)
```

asymptotic_ks_pvalue

Asymptotic Kolmogorov distribution survival function

```
asymptotic_ks_pvalue(q)
```

dp_anova

DP one-way ANOVA F-test

```
dp_anova(formula, data, epsilon, bounds, delta = NULL)
```

DP Regression & Inference

dp_lm

Differentially private linear regression

```
dp_lm(  
  formula,  
  data,  
  epsilon,  
  delta,  
  bounds = NULL,  
  assume_public_design = TRUE  
)
```

dp_lm_sensitivity

Global L2 sensitivity of OLS coefficients

```
dp_lm_sensitivity(  
  X,  
  y_bounds,  
  x_norm_bound = NULL,  
  assume_public_design = TRUE  
)
```

summary.dp_lm

Summarize a dp_lm fit

```
summary.dp_lm(object, level = 0.95, ...)
```

dp_glm

DP-SGD for generalized linear models

```
dp_glm(  
  formula,  
  data,  
  family = stats::gaussian(),  
  epsilon,  
  delta,  
  max_grad_norm = 1,  
  n_iter = 1000,
```

clip_gradients

Per-sample gradient clipping

```
clip_gradients(grads, max_grad_norm)
```

cpp_clip_gradients

Fast per-sample gradient clipping (C++)

```
cpp_clip_gradients(grads, max_grad_norm)
```

dp_confint

Privacy-aware confidence intervals

```
dp_confint(  
  object,  
  parm,  
  level = 0.95,  
  method = c("analytical", "parametric_bootstrap", "privacy_aware_bootstrap"),  
  B = 200,  
  ...  
)
```

confint.dp_lm

Confint interface for dp_lm objects

```
confint.dp_lm(object, parm, level = 0.95, method = "analytical", B = 200, ...)
```

Privacy Budget & Composition

new_privacy_budget

Create a privacy budget object

```
new_privacy_budget(  
  epsilon,  
  delta = 1e-06,  
  composition = c("basic", "advanced", "rdp")  
)
```

spend

Spend privacy budget

```
spend(budget, epsilon, delta = 0, description = "")
```

can_spend

Check whether a release fits in the remaining budget

```
can_spend(budget, epsilon, delta = 0)
```

basic_composition

Basic (sequential) composition

```
basic_composition(epsilons, deltas = rep(0, length(epsilons)))
```

advanced_composition_epsilon

Advanced composition

```
advanced_composition_epsilon(prior_epsilons, new_epsilon, delta_prime = 1e-06)
```

epsilon_to_rdp

Convert epsilon-DP to zCDP (RDP) parameter

```
epsilon_to_rdp(epsilon)
```

rdp_to_epsilon

Convert zCDP (RDP) parameter to (epsilon, delta)-DP

```
rdp_to_epsilon(rho, delta)
```

rdp_composition

RDP composition

```
rdp_composition(epsilons, delta = 1e-06)
```

laplace_plr_tail

Privacy loss random variables for the Laplace mechanism

```
laplace_plr_tail(t, epsilon)
laplace_plr_quantile(p, epsilon)
```

empirical_delta_from_losses

Empirical delta from simulated privacy losses

```
empirical_delta_from_losses(losses, epsilon)
```

simulate_gaussian_losses

Simulate privacy losses of Gaussian releases

```
simulate_gaussian_losses(n, sigma)
```

Diagnostics

validate_coverage

Validate confidence-interval coverage by Monte Carlo

```
validate_coverage(
  formula,
  true_betas,
  sigma = 1,
  data_gen = NULL,
  n = 500,
  n_sims = 200,
  epsilon = 1,
```

compare_utility

Compare utility of non-private and private fits

```
compare_utility(  
  formula,  
  data,  
  epsilon_grid,  
  delta = 1e-06,  
  bounds = NULL,  
  n_reps = 10  
)
```

compare_composition

Compare composition rules over a workflow

```
compare_composition(epsilons, delta = 1e-06)
```

dp_utility_diagnostics

Utility diagnostics for DP estimators

```
dp_utility_diagnostics(estimates, truth, ci_matrix = NULL)
```

Print Methods

print.dp_estimate

Print method for dp_estimate objects

```
print.dp_estimate(x, digits = 4, ...)
```

print.dp_htest

Print method for dp_htest objects

```
print.dp_htest(x, digits = 4, ...)
```

print.dp_lm

Print method for dp_lm objects

```
print.dp_lm(x, ...)
```

print.summary.dp_lm

Print method for summary.dp_lm objects

```
print.summary.dp_lm(x, digits = 4, ...)
```

print.dp_glm

Print method for dp_glm objects

```
print.dp_glm(x, ...)
```


print.dp_confint

Print method for dp_confint objects

```
print.dp_confint(x, digits = 4, ...)
```

print.privacy_budget

Print method for privacy_budget objects

```
print.privacy_budget(x, ...)
```

Generated from the package Rd documentation (DPrivStats 0.1.0). All DP releases consume declared privacy budget; outputs are valid under post-processing of privatized quantities.